CSE 4308 Database Lab - Viva Questions

Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE

Academic Year: AY25-26

1 Data Definition Language (DDL)

Q1. What happens if you try to DROP a table that is referenced by a foreign key in another table?

Answer: *Error is raised. You must drop the referencing table first or use DROP TABLE ... CASCADE to drop dependent objects.*

Q2. Explain the difference between `ON DELETE CASCADE` and `ON DELETE SET NULL`. When would each be inappropriate?

Answer: *CASCADE deletes child rows when parent is deleted. SET NULL sets FK to NULL. CASCADE inappropriate when child data should be preserved. SET NULL inappropriate when FK column is NOT NULL.*

Q3. Can you add a `NOT NULL` constraint to an existing column that already contains NULL values? What happens?

Answer: *No. The ALTER fails with an error. You must first UPDATE all NULL values to non-NULL before adding the constraint.*

Q4. What is the difference between a `UNIQUE` constraint and a `PRIMARY KEY`? Can a table have multiple of each?

Answer: *PRIMARY KEY = UNIQUE + NOT NULL. Table can have only one PK but multiple UNIQUE constraints. UNIQUE allows NULL (one per column in most DBs).*

Q5. If you `ALTER TABLE` to add a foreign key, but existing data violates the constraint, what happens?

Answer: *The ALTER fails with a constraint violation error. Existing data must satisfy the FK relationship before the constraint can be added.*

Q6. Can a foreign key reference a non-primary key column? What conditions must be met?

Answer: *Yes, but the referenced column must have a UNIQUE constraint. The FK must reference a column that guarantees uniqueness.*

Q7. What happens if you define a `CHECK` constraint that references another table's column?

Answer: *Error. CHECK constraints can only reference columns in the same row/table. Use triggers or FK constraints for cross-table validation.*

Q8. What is the difference between `DROP TABLE` and `DROP TABLE IF EXISTS`?

Answer: *DROP TABLE raises error if table doesn't exist. DROP TABLE IF EXISTS succeeds silently if table doesn't exist.*

Q9. Can you have multiple `UNIQUE` constraints on the same column? What about across multiple columns?

Answer: *Yes to both. Multiple UNIQUE on same column is redundant but allowed. UNIQUE(a,b) means combination must be unique, different from separate UNIQUE(a), UNIQUE(b).*

Q10. What is `ON UPDATE CASCADE`? When would you use it?

Answer: *When parent PK is updated, child FK values are automatically updated to match. Use when PK values may change and referential integrity must be maintained.*

Q11. Can you alter a column's data type if it contains data? What restrictions apply?

Answer: *Yes, if data is compatible with new type. Cannot change VARCHAR to INT if column contains non-numeric strings. May need explicit USING clause for conversion.*

Q12. What happens if you try to add a `PRIMARY KEY` to a column that has duplicate values?

Answer: *Error. PK requires uniqueness. Must remove duplicates first before adding the constraint.*

Q13. What is the difference between a composite primary key and multiple unique constraints?

Answer: *Composite PK: combination of columns is unique and identifies row. Multiple UNIQUE: each constraint enforces uniqueness independently. Composite PK(a,b) allows duplicate a if b differs.*

Q14. Can a foreign key reference a composite primary key? How do you define it?

Answer: *Yes. FK must include all columns of composite PK. Syntax: FOREIGN KEY (col1, col2) REFERENCES parent(pk1, pk2).*

Q15. You have table A with PK, table B with FK referencing A, and table C with FK referencing B. What order must you DROP them? What if you use CASCADE?

Answer: *Without CASCADE: C first, then B, then A (children before parents). With DROP TABLE A CASCADE: drops A and all dependent objects automatically.*

Q16. A column has DEFAULT 0 and NOT NULL. What happens if you INSERT without specifying this column?

Answer: *Column gets value 0 (the default). No error because NOT NULL is satisfied by the default value.*

2 Data Manipulation Language (DML)

Q17. Can you use a subquery in an `UPDATE` statement's `SET` clause? What restriction applies?

Answer: *Yes. The subquery must return exactly one value (scalar subquery). Multiple rows cause an error.*

Q18. If an `INSERT` violates a `UNIQUE` constraint, does it affect other rows in a multi-row insert?

Answer: *Depends on DB. PostgreSQL: entire statement fails, no rows inserted (atomic). Some DBs may insert valid rows before the error.*

Q19. What happens when you `UPDATE` a column that is part of a primary key and is referenced by foreign keys?

Answer: *Depends on FK definition. ON UPDATE CASCADE: child FKs updated. ON UPDATE RESTRICT/NO ACTION: error if children exist.*

Q20. Explain what happens if you run: `DELETE FROM employees WHERE department_id IN (SELECT id FROM departments WHERE budget < 0)` and the subquery returns no rows.

Answer: *No rows deleted. IN with empty set is always false, so WHERE condition matches nothing.*

Q21. Can you insert into a table without specifying column names? What are the risks?

Answer: *Yes: INSERT INTO t VALUES (...). Risk: breaks if table structure changes, wrong column order causes data in wrong columns or type errors.*

Q22. What is the difference between `INSERT INTO ... VALUES` and `INSERT INTO ... SELECT`?

Answer: *VALUES inserts literal values you specify. SELECT inserts results from a query, can insert multiple rows from another table.*

Q23. Can you `UPDATE` multiple columns in a single statement? Can they reference each other?

Answer: *Yes, multiple columns: SET a=1, b=2. They reference OLD values, not each other's new values. SET a=b, b=a swaps correctly.*

Q24. Can you use `LIMIT` in an `UPDATE` or `DELETE` statement? What does it do?

Answer: *PostgreSQL: No direct LIMIT, use subquery with ctid. MySQL: Yes. Limits number of rows affected. Useful for batch operations.*

Q25. Can you insert a row with only default values? What is the syntax?

Answer: *Yes. Syntax: INSERT INTO table DEFAULT VALUES; All columns get their default or NULL if no default defined.*

Q26. What happens if you try to insert a string into an integer column?

Answer: *Error if string is not numeric. If string is numeric like `'123'`, implicit conversion may succeed depending on DB settings.*

Q27. What is the difference between implicit and explicit type conversion in INSERT?

Answer: *Implicit: DB auto-converts compatible types (`'123'` to 123). Explicit: use CAST(x AS type). Implicit may fail or give unexpected results.*

Q28. You run UPDATE employees SET salary = salary * 1.1. The table has 1000 rows. How many rows are affected if 50 have NULL salary?

Answer: *1000 rows affected (all touched), but 50 remain NULL because NULL * 1.1 = NULL. Affected count includes unchanged rows.*

Q29. Write a single statement to delete all employees who earn more than their department's average salary.

Answer: *DELETE FROM employees e WHERE salary > (SELECT AVG(salary) FROM employees WHERE department_id = e.department_id). Uses correlated subquery.*

3 Joins

Q30. In a `LEFT JOIN`, if the right table has multiple matching rows, how many rows appear in the result?

Answer: *Multiple rows. Left table row is repeated for each matching right table row. Result has one row per match.*

Q31. What is the difference between `JOIN ... ON` and `JOIN ... USING`? When can you not use `USING`?

Answer: *ON: specify full condition (a.id = b.id). USING(col): columns must have same name in both tables. Can't use USING when column names differ.*

Q32. How does a `NATURAL JOIN` decide which columns to join on? What is the danger of using it?

Answer: *Joins on ALL columns with same name. Danger: adding a column with common name (like `'name'`, `'id'`) silently changes join behavior.*

Q33. If both tables in a join have NULL values in the join column, will those rows match?

Answer: *No. NULL = NULL is unknown/false. Rows with NULL in join column never match each other.*

Q34. Explain the difference between `WHERE` conditions and `ON` conditions in an outer join. Give an example where they produce different results.

Answer: *ON: applied during join, non-matches still appear with NULLs. WHERE: filters after join, removes non-matching rows. LEFT JOIN ... ON a.x=b.x WHERE b.y=1 removes unmatched left rows.*

Q35. What is a self-join? Give a scenario where it's necessary.

Answer: *Joining table to itself using aliases. Example: employees table with manager_id referencing employee_id. Self-join to get employee with their manager's name.*

Q36. In a `FULL OUTER JOIN`, how are NULLs represented for non-matching rows from each side?

Answer: *Non-matching left row: right columns are NULL. Non-matching right row: left columns are NULL. Shows all rows from both tables.*

Q37. Can you join more than two tables? What is the order of evaluation?

Answer: *Yes. Evaluated left to right: A JOIN B JOIN C = (A JOIN B) JOIN C. Use parentheses to control order if needed.*

Q38. What is a cross join? What is its result size if table A has M rows and table B has N rows?

Answer: *Cartesian product - every row of A paired with every row of B. Result has M * N rows.*

Q39. What is the difference between `INNER JOIN` and comma-separated tables in `FROM`?

Answer: *Functionally same if WHERE has join condition. INNER JOIN is explicit and clearer. Comma syntax is older, join condition in WHERE.*

Q40. Can you join a table with itself using different aliases? Why would you do this?

Answer: *Yes, that's a self-join. Use to compare rows within same table or follow hierarchical relationships (employee-manager).*

Q41. What happens if you omit the `ON` clause in an `INNER JOIN`?

Answer: *Syntax error in standard SQL. Some DBs treat it as cross join. Always specify ON condition.*

Q42. How do you find rows in table A that have no matching rows in table B?

Answer: *LEFT JOIN B ON ... WHERE B.key IS NULL. Or: WHERE NOT EXISTS (SELECT 1 FROM B WHERE B.key = A.key).*

Q43. If you join three tables A, B, C, does the order of joins matter for the result? For performance?

Answer: *Result: same for inner joins (mathematically). Performance: yes, optimizer chooses order but hints can help. Outer join order affects results.*

Q44. Table employees has 100 rows, departments has 5 rows. What is the maximum possible result size for: INNER JOIN? LEFT JOIN? CROSS JOIN?

Answer: *INNER: 100 max (if all match one dept). LEFT: 100 min (all left rows appear). CROSS: 500 exactly (100 * 5).*

Q45. Write a query to find employees whose salary is higher than all employees in department 10, using a join (not subquery).

Answer: *SELECT e1.* FROM employees e1 JOIN (SELECT MAX(salary) as max_sal FROM employees WHERE dept_id=10) d ON e1.salary > d.max_sal. Or self-join with GROUP BY.*

Q46. Explain why LEFT JOIN a ON condition WHERE a.col = value often gives unexpected results.

Answer: *WHERE filters AFTER join, removing rows where a.col is NULL (unmatched rows). Use AND in ON clause instead to preserve unmatched left rows.*

Q47. How do you find all employees along with their manager's name (including those without managers)?

Answer: *SELECT e.name, m.name as manager FROM employees e LEFT JOIN employees m ON e.manager_id = m.id. Self-join with LEFT keeps employees without managers.*

Q48. What is the difference between these: (A JOIN B) JOIN C versus A JOIN (B JOIN C)?

Answer: *For INNER joins, same result (associative). For OUTER joins, order can change results. Parentheses control which join happens first.*

Q49. Write a query to find pairs of employees who work in the same department.

Answer: *SELECT e1.name, e2.name FROM employees e1 JOIN employees e2 ON e1.dept_id = e2.dept_id AND e1.id < e2.id. Self-join, e1.id < e2.id avoids duplicates.*

Q50. How does USING differ from ON when column names are the same?

Answer: *USING(col) produces one col column in result. ON a.col = b.col produces two columns (a.col, b.col). USING is shorter when names match.*

Q51. What happens in a LEFT JOIN if the ON condition is always true (ON 1=1)?

Answer: *Becomes a cross join - every left row paired with every right row. All left rows appear, each multiple times.*

Q52. Write a query to find employees whose salary is higher than their department's average.

Answer: *SELECT e.* FROM employees e JOIN (SELECT dept_id, AVG(salary) avg_sal FROM employees GROUP BY dept_id) d ON e.dept_id = d.dept_id WHERE e.salary > d.avg_sal.*

Q53. What is the result if you RIGHT JOIN but the right table is empty?

Answer: *Result is empty. No rows from right table means no rows to join. Opposite of LEFT JOIN with empty left table.*

Q54. How do you find departments that have employees but no managers assigned?

Answer: *SELECT DISTINCT e.dept_id FROM employees e LEFT JOIN employees m ON e.manager_id = m.id WHERE e.manager_id IS NOT NULL AND m.id IS NULL. FK exists but no matching manager.*

Q55. Write a query to find customers and their orders, including customers with no orders and orders with no customers (orphans).

Answer: *SELECT * FROM customers c FULL OUTER JOIN orders o ON c.id = o.customer_id. FULL OUTER includes unmatched rows from both sides.*

Q56. Can you join the same table twice in one query? When would you need this?

Answer: *Yes. Example: SELECT * FROM orders o JOIN products p1 ON o.product1_id = p1.id JOIN products p2 ON o.product2_id = p2.id. Multiple FKs to same table.*

Q57. What is wrong with: SELECT * FROM A, B LEFT JOIN C ON B.id = C.b_id?

Answer: *Comma has lower precedence than JOIN. Parses as: A, (B LEFT JOIN C). Use explicit joins: A JOIN B ON ... LEFT JOIN C ON ...*

4 Aggregate Functions

Q58. What is the difference between `COUNT(*)` and `COUNT(column_name)`?

Answer: *COUNT(*) counts all rows including NULLs. COUNT(column) counts non-NULL values in that column only.*

Q59. How do `SUM()`, `AVG()`, `MIN()`, and `MAX()` handle NULL values?

Answer: *They ignore NULLs. SUM of 1,2,NULL = 3. AVG of 1,2,NULL = 1.5 (divides by 2, not 3).*

Q60. What does `COUNT(DISTINCT column)` do? How is it different from `COUNT(column)`?

Answer: *COUNT(DISTINCT) counts unique non-NULL values. COUNT(column) counts all non-NULL values including duplicates.*

Q61. If all values in a column are NULL, what does `AVG(column)` return?

Answer: *Returns NULL. No non-NULL values to average.*

Q62. Can you use an aggregate function in a `WHERE` clause? Why or why not?

Answer: *No. WHERE filters rows before grouping. Aggregates need grouped data. Use HAVING for aggregate conditions.*

Q63. What happens if you use `SUM()` on a VARCHAR column containing numeric strings?

Answer: *Error in strict mode. Some DBs attempt implicit conversion. PostgreSQL: error. Best practice: use proper numeric types.*

Q64. What is the difference between `COUNT(1)` and `COUNT(*)`?

Answer: *Functionally identical. COUNT(1) counts rows where 1 is not null (always). COUNT(*) counts all rows. Same result, same performance.*

Q65. What does `SUM(DISTINCT column)` do? When would you use it?

Answer: *Sums unique values only. Use when column has duplicates you want to count once, like summing distinct product prices.*

Q66. What is the result of `AVG(column)` if the column has one NULL and one non-NULL value?

Answer: *Returns the non-NULL value. NULL is ignored, so AVG of just that one value equals itself.*

Q67. How do you calculate the total count and distinct count in a single query?

Answer: *SELECT COUNT(*), COUNT(DISTINCT column) FROM table. Both aggregates in same SELECT.*

Q68. What is the difference between `MIN()` on strings vs numbers?

Answer: *Numbers: smallest numeric value. Strings: first in alphabetical/collation order. `'Apple'` < `'Banana'`. Case sensitivity depends on collation.*

Q69. Can you use `COUNT()` with a `CASE` expression inside? What does it count?

Answer: *Yes. COUNT(CASE WHEN condition THEN 1 END) counts rows matching condition. CASE returns NULL when false, which COUNT ignores.*

Q70. Table has values: 10, 20, 20, NULL, NULL. What is COUNT(*)? COUNT(col)? COUNT(DISTINCT col)? SUM(col)? AVG(col)?

Answer: *COUNT(*)=5, COUNT(col)=3, COUNT(DISTINCT col)=2, SUM(col)=50, AVG(col)=16.67 (50/3).*

Q71. How would you count employees per department AND show departments with zero employees?

Answer: *SELECT d.name, COUNT(e.id) FROM departments d LEFT JOIN employees e ON d.id=e.dept_id GROUP BY d.id, d.name. COUNT(e.id) gives 0 for empty depts.*

5 GROUP BY and HAVING

Q72. What is the difference between `WHERE` and `HAVING`? Can they be used together?

Answer: *WHERE filters rows before grouping. HAVING filters groups after aggregation. Yes, can use both: WHERE first, then GROUP BY, then HAVING.*

Q73. If you `GROUP BY` a column with NULL values, how are the NULLs grouped?

Answer: *All NULLs grouped together into one group. NULL is treated as equal to NULL for grouping purposes.*

Q74. Can you use a column alias defined in `SELECT` inside the `HAVING` clause?

Answer: *PostgreSQL/MySQL: Yes. Standard SQL: No. Best practice: repeat expression or use subquery for portability.*

Q75. What happens if you select a non-aggregated column that is not in the `GROUP BY` clause?

Answer: *PostgreSQL: Error. MySQL (non-strict): Returns arbitrary value from group. Always include non-aggregated columns in GROUP BY.*

Q76. Can you `GROUP BY` an expression or only column names?

Answer: *Yes, can GROUP BY expressions: GROUP BY EXTRACT(YEAR FROM date), GROUP BY salary/1000. Same expression should be in SELECT.*

Q77. Explain the order of execution: `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`.

Answer: *FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY. WHERE before grouping, HAVING after, SELECT evaluates aliases, ORDER BY last.*

Q78. Can you use `HAVING` without `GROUP BY`? What does it do?

Answer: *Yes. Treats entire table as one group. HAVING condition applies to aggregates over all rows. SELECT COUNT(*) HAVING COUNT(*) > 10.*

Q79. What is the difference between `GROUP BY column` and `GROUP BY 1`?

Answer: *GROUP BY 1 references first column in SELECT list by position. GROUP BY column uses explicit name. Position reference is less readable.*

Q80. Can you `GROUP BY` multiple columns? How does the grouping work?

Answer: *Yes. Creates groups for each unique combination of values. GROUP BY dept, year creates separate group for each (dept, year) pair.*

Q81. Can you use a subquery in the `HAVING` clause?

Answer: *Yes. HAVING COUNT(*) > (SELECT AVG(cnt) FROM ...). Subquery must return scalar value for comparison.*

Q82. What happens if `HAVING` references a column not in `GROUP BY` or an aggregate?

Answer: *Error. HAVING can only reference GROUP BY columns or aggregate expressions. Non-grouped columns are ambiguous.*

Q83. How do you filter groups to show only those with exactly one row?

Answer: *HAVING COUNT(*) = 1. Groups where aggregate count equals one.*

Q84. Write a query to find departments where the highest salary is more than twice the lowest salary.

Answer: *SELECT dept_id FROM employees GROUP BY dept_id HAVING MAX(salary) > 2 * MIN(salary).*

Q85. You want the top 3 departments by employee count. Write the query.

Answer: *SELECT dept_id, COUNT(*) as cnt FROM employees GROUP BY dept_id ORDER BY cnt DESC LIMIT 3.*

Q86. How do you find duplicate rows in a table based on columns (name, email)?

Answer: *SELECT name, email, COUNT(*) FROM users GROUP BY name, email HAVING COUNT(*) > 1.*

6 Views

Q87. Can you `INSERT` into a view? What conditions must the view satisfy?

Answer: *Yes, if view is updatable: single table, no aggregates, no DISTINCT, no GROUP BY, no joins, includes all NOT NULL columns without defaults.*

Q88. If the underlying table of a view is dropped, what happens to the view?

Answer: *View becomes invalid. Querying it raises error. View definition remains but is broken until table is recreated.*

Q89. Can you create a view based on another view? What are the implications?

Answer: *Yes, views can reference other views. Creates dependency chain. Performance may suffer due to nested query expansion. Changes cascade.*

Q90. What makes a view non-updatable? List at least 4 conditions.

Answer: *1) Joins multiple tables 2) Has GROUP BY/aggregates 3) Has DISTINCT 4) Has UNION 5) Has subqueries in SELECT 6) Missing required NOT NULL columns.*

Q91. What is a materialized view? How does it differ from a regular view in terms of storage and performance?

Answer: *Materialized view stores query result physically. Regular view re-executes query each time. Materialized: faster reads, needs refresh. Regular: always current, slower.*

Q92. If you update data through an updatable view, does the underlying table change?

Answer: *Yes. View is just a window to table. UPDATE/INSERT/DELETE through view modifies actual base table data.*

Q93. Can a view have an `ORDER BY` clause? What restrictions apply?

Answer: *PostgreSQL: Yes, but not guaranteed without ORDER BY in outer query. Standard SQL: No, unless with LIMIT/OFFSET. Views represent sets, inherently unordered.*

Q94. What is `WITH CHECK OPTION` in a view definition?

Answer: *Prevents INSERT/UPDATE that would make row invisible to view. If view has WHERE salary > 50000, can't insert row with salary 40000.*

Q95. What happens if you alter the underlying table's structure after creating a view?

Answer: *Depends on change. Adding column: view unaffected. Dropping column used by view: view becomes invalid. Renaming: view may break.*

Q96. Can a view join multiple tables? Is such a view updatable?

Answer: *Yes, view can join tables. Generally not updatable. Some DBs allow limited updates to one table if key preserved.*

Q97. What is the difference between `CREATE VIEW` and `CREATE OR REPLACE VIEW`?

Answer: *CREATE fails if view exists. CREATE OR REPLACE overwrites existing view, preserving grants/permissions. Safer for updates.*

Q98. How do you refresh a materialized view? What happens during refresh?

Answer: *REFRESH MATERIALIZED VIEW name. Query re-executes, results replace old data. CONCURRENTLY option allows reads during refresh.*

Q99. Can a view have parameters like a function?

Answer: *No. Views don't accept parameters. Use functions returning tables or use session variables as workaround.*

Q100. You create a view: CREATE VIEW high_earners AS SELECT * FROM employees WHERE salary > 50000. Can you INSERT (1, `'John'`, 30000)?

Answer: *Without WITH CHECK OPTION: Yes, row inserted but invisible through view. With CHECK OPTION: Error, row violates view condition.*

Q101. A view selects columns a, b from table t. You ALTER TABLE t ADD COLUMN c. Does SELECT * FROM view show column c?

Answer: *No. View was defined with specific columns (a, b). New column c not included. Must recreate view to include new columns.*

7 Roles and Privileges (DCL)

Q102. What is the difference between `GRANT SELECT ON table TO user` and `GRANT SELECT ON table TO user WITH GRANT OPTION`?

Answer: *First gives SELECT privilege only. WITH GRANT OPTION allows user to grant that privilege to others.*

Q103. If you revoke a privilege from a user who granted it to others, what happens to those grants?

Answer: *CASCADE: all derived grants revoked. RESTRICT: revoke fails if others depend on it. Default behavior varies by DB.*

Q104. Can a role be granted to another role? How does privilege inheritance work?

Answer: *Yes. Role inherits all privileges of granted roles. User with role A, where A has role B, gets privileges of both A and B.*

Q105. What is the difference between revoking a privilege and denying a privilege?

Answer: *REVOKE removes granted privilege. DENY (SQL Server) explicitly blocks, overrides grants. PostgreSQL lacks DENY; use careful grant management.*

Q106. What privileges are needed to create a view that references multiple tables owned by different users?

Answer: *Need SELECT privilege on all referenced tables. View creator must have access to all underlying objects.*

Q107. What is the difference between `GRANT ALL` and granting individual privileges?

Answer: *GRANT ALL gives all applicable privileges (SELECT, INSERT, UPDATE, DELETE, etc.). Individual grants give specific privileges only, more secure.*

Q108. Can you grant privileges on specific columns rather than entire tables?

Answer: *Yes. GRANT SELECT (col1, col2) ON table TO user. User can only access specified columns.*

Q109. What is `PUBLIC` in the context of grants? What are the risks?

Answer: *PUBLIC is implicit role including all users. GRANT TO PUBLIC gives access to everyone, including future users. Security risk for sensitive data.*

Q110. How do you revoke only the `WITH GRANT OPTION` without revoking the privilege itself?

Answer: *REVOKE GRANT OPTION FOR SELECT ON table FROM user. User keeps SELECT but can't grant it to others.*

Q111. What privileges are needed to execute a stored procedure?

Answer: *EXECUTE privilege on the procedure. May also need privileges on objects accessed within procedure, depending on SECURITY DEFINER/INVOKER.*

Q112. Can you grant `SELECT` on a view without granting access to underlying tables?

Answer: *Yes. View acts as security layer. Grant SELECT on view, not on base tables. User sees only what view exposes.*

Q113. User A grants SELECT to User B WITH GRANT OPTION. B grants to C. You revoke from B. What happens to C's access?

Answer: *With CASCADE (default): C loses access too. With RESTRICT: Revoke fails because C depends on B's grant.*

8 Subqueries

Q114. What is the difference between a correlated and non-correlated subquery? Which is typically faster and why?

Answer: *Non-correlated: independent, runs once. Correlated: references outer query, runs per outer row. Non-correlated faster due to single execution.*

Q115. When must a subquery return exactly one value (scalar subquery)?

Answer: *When used in SELECT list, in comparison (=, <, >), in SET clause of UPDATE. Multi-row result causes error.*

Q116. What is the difference between `IN` and `EXISTS`? When would you prefer one over the other?

Answer: *IN: checks value membership in list. EXISTS: checks if subquery returns any rows. EXISTS better for large subquery results, stops at first match.*

Q117. Can a subquery reference columns from the outer query? What is this called?

Answer: *Yes. Called correlated subquery. Inner query depends on outer query values, re-evaluated for each outer row.*

Q118. What happens if a subquery in a `WHERE column = (subquery)` returns multiple rows?

Answer: *Error. Scalar comparison requires single value. Use IN, ANY, or ALL for multiple values.*

Q119. Can you use a subquery in the `FROM` clause? What is this called?

Answer: *Yes. Called derived table or inline view. Must have alias: SELECT * FROM (SELECT ...) AS alias.*

Q120. How does `NOT IN` behave when the subquery result contains NULL values?

Answer: *Returns no rows if NULL present. x NOT IN (1, NULL) is UNKNOWN for any x. Use NOT EXISTS instead to avoid this.*

Q121. Can you use aggregate functions in a subquery? Can the outer query reference the alias of that aggregate?

Answer: *Yes, aggregates work in subquery. No, outer can't reference inner aliases directly. Return aggregate as column, reference via derived table.*

Q122. What is the difference between `ANY` and `ALL` with subqueries?

Answer: *ANY: true if condition holds for at least one value. ALL: true if condition holds for every value. x > ANY(1,2,3): x>1. x > ALL(1,2,3): x>3.*

Q123. Can you use `ORDER BY` in a subquery? When would you need to?

Answer: *Yes, but usually meaningless unless with LIMIT. Subquery in FROM can use ORDER BY with LIMIT to get top N before joining.*

Q124. Can a subquery in `SELECT` return multiple columns?

Answer: *No. SELECT list subquery must return single value. Use JOIN or derived table for multiple columns.*

Q125. How do you rewrite a correlated subquery as a join? Is it always possible?

Answer: *Often possible using JOIN with same condition. WHERE EXISTS (SELECT 1 FROM b WHERE b.x=a.x) becomes JOIN b ON b.x=a.x. Not always cleaner.*

Q126. What happens if `EXISTS (SELECT NULL)`? Does it return true or false?

Answer: *Returns TRUE. EXISTS checks if any row exists, not the value. SELECT NULL still returns a row, so EXISTS is true.*

Q127. Can you use `LIMIT` inside a subquery?

Answer: *Yes. Useful for getting top N in derived table: SELECT * FROM (SELECT * FROM t ORDER BY x LIMIT 5) sub.*

Q128. What is a common table expression (CTE)? How is it different from a subquery?

Answer: *CTE: WITH name AS (query) SELECT ... Named, reusable in same statement. Subquery: inline, can't reference multiple times. CTE more readable.*

Q129. Write a query to find employees who earn more than the average salary of their own department.

Answer: *SELECT * FROM employees e WHERE salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id). Correlated subquery.*

Q130. What's wrong with: SELECT * FROM employees WHERE dept_id NOT IN (SELECT dept_id FROM departments WHERE budget IS NULL)?

Answer: *If any dept_id in subquery is NULL, NOT IN returns no rows. NULL in the list makes all comparisons UNKNOWN.*

Q131. Rewrite using CTE: SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees).

Answer: *WITH avg_sal AS (SELECT AVG(salary) as avg FROM employees) SELECT * FROM employees, avg_sal WHERE salary > avg_sal.avg.*

9 String Functions

Q132. What does `SUBSTRING('Hello', 2, 3)` return? Are indices 0-based or 1-based?

Answer: *Returns `'ell'`. SQL strings are 1-based. Position 2 = `'e'`, length 3 = `'ell'`.*

Q133. What is the difference between `CHAR_LENGTH()` and `LENGTH()` for multibyte characters?

Answer: *CHAR_LENGTH: counts characters. LENGTH: counts bytes. For UTF-8 multibyte char, CHAR_LENGTH=1 but LENGTH may be 2-4 bytes.*

Q134. How does the `LIKE` pattern `'%_%'` differ from `'%%'`?

Answer: *'%_%' requires at least one character (underscore matches exactly one). `'%%'` matches anything including empty string.*

Q135. What happens if you concatenate a string with NULL using `CONCAT()` vs the `||` operator?

Answer: *CONCAT ignores NULL: CONCAT(`'a'`,NULL,`'b'`)=`'ab'`. || propagates NULL: `'a'`||NULL||`'b'`=NULL. Use COALESCE with || for safety.*

Q136. What does `TRIM()` do by default? How do you trim only leading or only trailing spaces?

Answer: *Default: removes both leading and trailing spaces. TRIM(LEADING FROM str), TRIM(TRAILING FROM str), or LTRIM/RTRIM functions.*

Q137. What is the difference between `UPPER()` and `INITCAP()`?

Answer: *UPPER: all characters uppercase. INITCAP: first letter of each word uppercase, rest lowercase. `'heLLo wORLD'` -> `'Hello World'`.*

Q138. How do you escape a literal `%` or `_` in a `LIKE` pattern?

Answer: *Use ESCAPE clause: WHERE col LIKE `'100\%'` ESCAPE `'\'`. Or use bracket in some DBs: [%], [_].*

Q139. What does `POSITION('b' IN 'abc')` return? What if the substring is not found?

Answer: *Returns 2 (1-based position). Returns 0 if not found, not NULL.*

Q140. How do you pad a string to a fixed length? What functions are used?

Answer: *LPAD(str, length, padchar): pad left. RPAD(str, length, padchar): pad right. LPAD(`'5'`, 3, `'0'`) = `'005'`.*

Q141. What does `COALESCE(NULL, '', 'default')` return? Why?

Answer: *Returns '' (empty string). COALESCE returns first non-NULL. Empty string is not NULL, so it's returned.*

Q142. Write a query to find names that start with `'A'` and end with `'n'`.

Answer: *SELECT name FROM employees WHERE name LIKE `'A%n'`. % matches any characters between A and n.*

Q143. How do you extract the domain from an email address `'user@domain.com'`?

Answer: *SELECT SUBSTRING(email FROM POSITION(`'@'` IN email) + 1). Or: SPLIT_PART(email, `'@'`, 2) in PostgreSQL.*

Q144. What does `LIKE '___'` match? How is it different from `LIKE '%'`?

Answer: *'_ _ _' matches exactly 3 characters. '%' matches zero or more characters. '_ _ _' won't match `'AB'` or `'ABCD'`.*

Q145. Write a pattern to match strings that contain exactly one `'a'` anywhere.

Answer: *Not directly possible with LIKE alone. Use: WHERE name LIKE `'%a%'` AND name NOT LIKE `'%a%a%'`. Or use regexp.*

Q146. What does `LIKE 'a%b%c'` match? Give examples of matching and non-matching strings.

Answer: *Matches strings starting with `'a'`, containing `'b'` later, ending with `'c'` after that. `'abc'`, `'aXbYc'` match. `'ab'`, `'bac'`, `'acb'` don't match.*

Q147. How do you match a literal underscore character in a LIKE pattern?

Answer: *Use ESCAPE: WHERE col LIKE `'%\_%' ESCAPE '\'`. The backslash makes _ literal, not wildcard.*

Q148. What is the difference between `LIKE 'abc'` and `= 'abc'`?

Answer: *Functionally same when no wildcards. LIKE may be slower (pattern matching). = is direct comparison. Use = when exact match needed.*

Q149. Write a query to find products whose code is exactly 5 characters and starts with `'P'`.

Answer: *SELECT * FROM products WHERE code LIKE `'P_ _ _ _'`. P followed by exactly 4 underscores = 5 chars total starting with P.*

Q150. How does LIKE handle case sensitivity? How do you make it case-insensitive?

Answer: *LIKE is case-sensitive by default. Use ILIKE (PostgreSQL) for case-insensitive. Or: WHERE LOWER(col) LIKE LOWER(`'Pattern'`).*

Q151. Write a pattern to match phone numbers in format XXX-XXX-XXXX.

Answer: *WHERE phone LIKE `'_ _ _-_ _ _-_ _ _ _'`. Three chars, dash, three chars, dash, four chars. Won't validate digits only.*

Q152. What does `LIKE '%'` match? Does it match NULL? Does it match empty string?

Answer: *Matches any string including empty string. Does NOT match NULL (nothing equals NULL). Empty string matches.*

Q153. How do you find names that do NOT start with a vowel?

Answer: *WHERE name NOT LIKE `'A%'` AND name NOT LIKE `'E%'` AND name NOT LIKE `'I%'` AND name NOT LIKE `'O%'` AND name NOT LIKE `'U%'`. Or use regexp.*

Q154. What is the result of `REPLACE('hello', 'l', 'L')`? Does it replace all occurrences?

Answer: *Returns `'heLLo'`. Yes, REPLACE replaces all occurrences, not just first.*

Q155. How do you reverse a string in PostgreSQL?

Answer: *REVERSE(`'hello'`) returns `'olleh'`. Built-in function in PostgreSQL.*

Q156. What does `REPEAT('ab', 3)` return?

Answer: *Returns `'ababab'`. Repeats the string 3 times.*

Q157. How do you find the position of the second occurrence of `'a'` in a string?

Answer: *POSITION(`'a'` IN SUBSTRING(str FROM POSITION(`'a'` IN str) + 1)) + POSITION(`'a'` IN str). Find first, search from after it.*

Q158. What is the difference between `LPAD('5', 3, '0')` and `RPAD('5', 3, '0')`?

Answer: *LPAD: pads left, returns `'005'`. RPAD: pads right, returns `'500'`. Both make string 3 chars long.*

Q159. How do you extract the file extension from `'document.pdf'`?

Answer: *SUBSTRING(filename FROM POSITION(`'.'` IN filename) + 1). Or: SPLIT_PART(filename, `'.'`, 2). Returns `'pdf'`.*

Q160. What does `LEFT('PostgreSQL', 4)` return? What about `RIGHT('PostgreSQL', 3)`?

Answer: *LEFT returns `'Post'` (first 4 chars). RIGHT returns `'SQL'` (last 3 chars).*

10 Date Functions

Q161. How do you extract the month from a date column? What does it return (number or name)?

Answer: *EXTRACT(MONTH FROM date) returns number (1-12). Use TO_CHAR(date, `'Month'`) for name.*

Q162. What is the result type of subtracting two dates?

Answer: *PostgreSQL: interval type showing days/time difference. Some DBs return integer (days).*

Q163. How do you add 3 months to a date? What happens if the resulting day doesn't exist (e.g., Jan 31 + 1 month)?

Answer: *date + INTERVAL `'3 months'`. Jan 31 + 1 month = Feb 28/29 (last day of month). Overflow handled by capping to month end.*

Q164. What is the difference between `CURRENT_DATE`, `CURRENT_TIMESTAMP`, and `NOW()`?

Answer: *CURRENT_DATE: date only (no time). CURRENT_TIMESTAMP/NOW(): date and time. NOW() is PostgreSQL function, CURRENT_TIMESTAMP is standard.*

Q165. How do you find all records from the last 30 days?

Answer: *WHERE date_column >= CURRENT_DATE - INTERVAL `'30 days'`. Or: >= NOW() - INTERVAL `'30 days'` for timestamp.*

Q166. What is the difference between `DATE` and `TIMESTAMP` data types?

Answer: *DATE: year, month, day only. TIMESTAMP: includes hours, minutes, seconds, fractional seconds. TIMESTAMP uses more storage.*

Q167. What does `AGE(date1, date2)` return in PostgreSQL?

Answer: *Returns interval showing difference in years, months, days. AGE(`'2024-01-15'`, `'2020-06-10'`) = `'3 years 7 mons 5 days'`.*

Q168. How do you get the day of the week from a date? What format is returned?

Answer: *EXTRACT(DOW FROM date): 0-6 (Sunday=0). EXTRACT(ISODOW FROM date): 1-7 (Monday=1). TO_CHAR(date,`'Day'`) for name.*

Q169. What is an interval? How do you add an interval to a date?

Answer: *Interval: duration of time (days, hours, etc.). Add with +: date + INTERVAL '1 year 2 months'. Result is new date/timestamp.*

Q170. How do you calculate the number of days between two dates?

Answer: *date1 - date2 gives interval. EXTRACT(DAY FROM (date1 - date2)) or cast to integer: (date1 - date2)::integer in PostgreSQL.*

Q171. Write a query to find employees hired in the current month of any year.

Answer: *SELECT * FROM employees WHERE EXTRACT(MONTH FROM hire_date) = EXTRACT(MONTH FROM CURRENT_DATE).*

Q172. How do you find the first day of the current month?

Answer: *DATE_TRUNC('month', CURRENT_DATE) or CURRENT_DATE - INTERVAL '1 day' * (EXTRACT(DAY FROM CURRENT_DATE) - 1).*

11 Set Operations

Q173. What is the difference between `UNION` and `UNION ALL`?

Answer: *UNION removes duplicates (slower, sorts). UNION ALL keeps all rows including duplicates (faster). Use UNION ALL when duplicates OK.*

Q174. What conditions must two queries satisfy to be combined with `UNION`?

Answer: *Same number of columns. Compatible data types in corresponding positions. Column names from first query used in result.*

Q175. How do `INTERSECT` and `EXCEPT` handle duplicate rows?

Answer: *Both remove duplicates by default. INTERSECT: rows in both queries. EXCEPT: rows in first but not second. Use ALL variant to keep duplicates.*

Q176. What is the precedence of set operations? How do you control evaluation order?

Answer: *INTERSECT has higher precedence than UNION/EXCEPT. Use parentheses to control: (A UNION B) INTERSECT C differs from A UNION (B INTERSECT C).*

Q177. How do set operations handle NULL values when determining duplicates?

Answer: *Two NULLs are considered equal for duplicate elimination. Unlike comparisons, NULL=NULL is true for UNION/INTERSECT/EXCEPT.*

Q178. Can you use `ORDER BY` with set operations? Where does it go?

Answer: *Yes, at the end of entire statement, applies to final result: SELECT ... UNION SELECT ... ORDER BY column.*

Q179. What column names appear in the result of a `UNION`?

Answer: *Column names from first SELECT query are used in the result set.*

Q180. Can you combine more than two queries with set operations?

Answer: *Yes. A UNION B UNION C. Evaluated left to right unless parentheses change order.*

Q181. What is the difference between `EXCEPT` and `NOT IN`?

Answer: *EXCEPT: set operation, removes duplicates, handles NULL properly. NOT IN: filters, keeps duplicates, fails with NULL in subquery.*

Q182. Can the queries in a `UNION` have different column data types?

Answer: *Types must be compatible. DB attempts implicit conversion. INT and BIGINT OK. VARCHAR and INT may fail or give unexpected results.*

Q183. Write a query to find customers who placed orders in 2023 but NOT in 2024.

Answer: *SELECT customer_id FROM orders WHERE EXTRACT(YEAR FROM order_date)=2023 EXCEPT SELECT customer_id FROM orders WHERE EXTRACT(YEAR FROM order_date)=2024.*

Q184. How do you find values that exist in both tables A and B?

Answer: *SELECT col FROM A INTERSECT SELECT col FROM B. Returns only values present in both result sets.*

Q185. What happens if you use UNION on queries with different column names but same types?

Answer: *Works fine. Result uses column names from first query. Data types must be compatible, names don't need to match.*

Q186. Can you use WHERE clause in each query of a UNION? What about ORDER BY?

Answer: *WHERE: yes, each SELECT can have its own WHERE. ORDER BY: only at end, applies to final combined result.*

Q187. How do you combine employees and contractors into one list, keeping duplicates?

Answer: *SELECT name, `'employee'` as type FROM employees UNION ALL SELECT name, `'contractor'` FROM contractors. UNION ALL keeps all rows.*

Q188. Write a query to find products that are either out of stock OR discontinued.

Answer: *SELECT product_id FROM inventory WHERE quantity = 0 UNION SELECT product_id FROM products WHERE discontinued = true.*

Q189. What is the difference between EXCEPT and MINUS?

Answer: *Same operation, different names. EXCEPT is standard SQL (PostgreSQL). MINUS is Oracle syntax. Both return rows in first query but not second.*

Q190. Can you use LIMIT with set operations? Where does it go?

Answer: *Yes, at end after all set operations. SELECT ... UNION SELECT ... LIMIT 10. Limits final combined result.*

Q191. How do you find departments that have no employees using set operations?

Answer: *SELECT id FROM departments EXCEPT SELECT DISTINCT dept_id FROM employees. Returns dept IDs with no matching employees.*

Q192. What is the result of: SELECT 1 UNION SELECT 1 UNION ALL SELECT 1?

Answer: *Returns two rows: 1 and 1. First UNION removes duplicate, then UNION ALL adds another 1. Order matters.*

12 NULL Handling

Q193. What is the result of `NULL = NULL`? How do you correctly check for NULL?

Answer: *NULL = NULL is UNKNOWN (not TRUE). Use IS NULL or IS NOT NULL to check for NULL values.*

Q194. What does `COALESCE(a, b, c)` return if `a` is NULL but `b` is not?

Answer: *Returns b. COALESCE returns first non-NULL argument from left to right.*

Q195. How does `ORDER BY` handle NULL values? Is it consistent across databases?

Answer: *PostgreSQL: NULLs last in ASC, first in DESC. MySQL: NULLs first in ASC. Use NULLS FIRST/LAST to control explicitly.*

Q196. What is the result of `NULL + 5`? What about `NULL || 'text'`?

Answer: *Both return NULL. Any arithmetic or concatenation with NULL produces NULL (NULL propagates).*

Q197. How do you sort NULLs first or last explicitly?

Answer: *ORDER BY column ASC NULLS FIRST, or ORDER BY column DESC NULLS LAST. Explicit control over NULL position.*

Q198. What is the result of `NOT NULL`?

Answer: *Returns NULL. NOT of unknown value is unknown. NOT NULL = NULL.*

Q199. How does `DISTINCT` treat multiple NULL values?

Answer: *All NULLs are grouped together as one distinct value. SELECT DISTINCT returns only one NULL.*

Q200. What is the result of `NULL IN (1, 2, NULL)`?

Answer: *Returns UNKNOWN (NULL). Can't determine if NULL equals 1, 2, or the NULL in the list.*

Q201. How do you replace NULL with a default value in the result?

Answer: *COALESCE(column, `'default'`) or CASE WHEN column IS NULL THEN `'default'` ELSE column END.*

Q202. What is the result of: WHERE NOT (salary > 50000) when salary is NULL?

Answer: *Row excluded. salary > 50000 is UNKNOWN, NOT UNKNOWN is UNKNOWN, WHERE filters out UNKNOWN.*

Q203. How do you check if two columns are equal, treating NULLs as equal?

Answer: *WHERE (a = b) OR (a IS NULL AND b IS NULL). Or use IS NOT DISTINCT FROM in PostgreSQL: a IS NOT DISTINCT FROM b.*

13 PL/SQL / PL/pgSQL

Q204. What is the difference between `DECLARE`, `BEGIN`, and `END` blocks?

Answer: *DECLARE: variable declarations section. BEGIN...END: executable code block. Structure: DECLARE vars; BEGIN code; END;.*

Q205. What happens if a `SELECT INTO` returns no rows? What about multiple rows?

Answer: *No rows: NO_DATA_FOUND exception (or FOUND=false). Multiple rows: TOO_MANY_ROWS exception. Use LIMIT 1 or handle exceptions.*

Q206. How do you handle exceptions in PL/pgSQL? What is the syntax?

Answer: *BEGIN ... EXCEPTION WHEN condition THEN handler; END. Example: EXCEPTION WHEN NO_DATA_FOUND THEN RAISE NOTICE `'Not found'`.*

Q207. What is the scope of a variable declared inside a nested block?

Answer: *Visible only within that block and its nested blocks. Outer block cannot access inner block's variables.*

Q208. What is the difference between `LOOP`, `WHILE`, and `FOR` loops?

Answer: *LOOP: infinite until EXIT. WHILE: condition checked before each iteration. FOR: iterates over range or query result, counter automatic.*

Q209. How do you exit a loop early? What is the syntax?

Answer: *EXIT; exits innermost loop. EXIT WHEN condition; exits when condition true. EXIT label; exits named outer loop.*

Q210. How do you raise a custom error/exception?

Answer: *RAISE EXCEPTION `'Error message %'`, variable; Stops execution and returns error to caller.*

Q211. What is the difference between `RAISE NOTICE` and `RAISE EXCEPTION`?

Answer: *NOTICE: informational message, execution continues. EXCEPTION: error, stops execution and rolls back transaction.*

Q212. Can you have nested blocks? What happens to variables in nested blocks?

Answer: *Yes. Inner block can access outer variables. Inner block can declare same-name variable, shadows outer. Changes to outer vars persist.*

Q213. What is `FOUND` in PL/pgSQL? When is it useful?

Answer: *Boolean set by SQL statements. True if last SELECT INTO/UPDATE/DELETE affected rows. Check after statement: IF NOT FOUND THEN...*

Q214. Write a PL/pgSQL block that doubles all salaries under 30000 and returns the count of updated rows.

Answer: *DO $$ DECLARE cnt INT; BEGIN UPDATE employees SET salary=salary*2 WHERE salary<30000; GET DIAGNOSTICS cnt = ROW_COUNT; RAISE NOTICE `'Updated %'`, cnt; END $$.*

Q215. What happens if you don't handle an exception in PL/pgSQL?

Answer: *Exception propagates to caller. If uncaught, transaction aborts and rolls back. Error message returned to client.*

14 Functions

Q216. What is the difference between `RETURNS` and `RETURN` in a function definition?

Answer: *RETURNS: declares return type in function signature. RETURN: statement inside function body that returns value.*

Q217. Can a function return multiple values? How?

Answer: *Yes. Use OUT parameters, RETURNS TABLE, or RETURNS SETOF. Multiple OUT params become record. TABLE/SETOF return multiple rows.*

Q218. Can a function modify data (INSERT, UPDATE, DELETE)? Should it?

Answer: *Yes, possible. Generally discouraged; use procedures for data modification. Functions should compute/return values. Side effects reduce clarity.*

Q219. What happens if a function does not execute a `RETURN` statement?

Answer: *Error at runtime: "control reached end of function without RETURN". All code paths must return a value.*

Q220. How do you call a function that returns a table/set of rows?

Answer: *SELECT * FROM function_name(). Or in FROM clause: SELECT col FROM table, function_name() f WHERE...*

Q221. What is `RETURNS SETOF` vs `RETURNS TABLE`?

Answer: *SETOF typename: returns multiple rows of existing type. TABLE(col1 type1, col2 type2): defines inline return structure. TABLE more flexible.*

Q222. Can a function have default parameter values?

Answer: *Yes. CREATE FUNCTION f(a INT DEFAULT 10). Call without arg uses default: f() uses a=10.*

Q223. Can you call a function inside a `CHECK` constraint?

Answer: *Limited. Only IMMUTABLE functions allowed in CHECK. Function must not access database or have side effects.*

Q224. What is `LANGUAGE SQL` vs `LANGUAGE plpgsql`?

Answer: *SQL: simple SQL statements, inlined by optimizer, no variables/control flow. plpgsql: full procedural language, variables, loops, conditions.*

Q225. Write a function that returns the full name by concatenating first_name and last_name with a space.

Answer: *CREATE FUNCTION full_name(f TEXT, l TEXT) RETURNS TEXT AS $$ SELECT f || ' ' || l; $$ LANGUAGE SQL.*

Q226. What is the difference between SECURITY DEFINER and SECURITY INVOKER functions?

Answer: *DEFINER: runs with privileges of function creator. INVOKER: runs with privileges of caller. DEFINER can allow controlled access to data.*

15 Procedures

Q227. What is the difference between a function and a procedure?

Answer: *Function: returns value, callable in expressions/SELECT. Procedure: no return value, called with CALL, can control transactions (COMMIT/ROLLBACK).*

Q228. Explain `IN`, `OUT`, and `INOUT` parameters with examples.

Answer: *IN: input only (default). OUT: output only, set inside procedure. INOUT: input and output, initial value passed, modified value returned.*

Q229. Can a procedure return a value? How is it different from a function's return?

Answer: *Procedure uses OUT parameters, not RETURN statement. Caller accesses OUT values. Function uses RETURN for single value in expressions.*

Q230. How do you call a procedure? Can you call it in a `SELECT` statement?

Answer: *CALL procedure_name(args). Cannot use in SELECT. Procedures are statements, not expressions.*

Q231. Can a procedure call another procedure? Can it call itself (recursion)?

Answer: *Yes to both. CALL other_procedure() inside procedure. Recursion works with proper exit condition.*

Q232. Can a procedure contain transaction control statements (`COMMIT`, `ROLLBACK`)?

Answer: *Yes, procedures can COMMIT/ROLLBACK. Functions cannot. This is key difference. Allows multi-statement transactions in procedures.*

Q233. What happens if an error occurs inside a procedure?

Answer: *Depends on exception handling. Without handler: procedure aborts, transaction rolled back, error raised to caller.*

Q234. Can you have multiple `OUT` parameters in a procedure?

Answer: *Yes. CREATE PROCEDURE p(OUT x INT, OUT y INT). Both values set inside and accessible to caller after CALL.*

Q235. How do you get the value of an `OUT` parameter after calling a procedure?

Answer: *CALL procedure(input_val, output_var). After call, output_var contains the OUT value. In psql: use DO block to capture.*

Q236. What is the difference between `CALL procedure()` and `SELECT procedure()`?

Answer: *CALL: correct way to invoke procedure. SELECT: used for functions. Procedures cannot be called with SELECT.*

Q237. Write a procedure that transfers amount from one account to another, with proper error handling.

Answer: *CREATE PROCEDURE transfer(from_id INT, to_id INT, amt DECIMAL) AS $$ BEGIN UPDATE accounts SET balance=balance-amt WHERE id=from_id; UPDATE accounts SET balance=balance+amt WHERE id=to_id; END $$ LANGUAGE plpgsql.*

16 JDBC

Q238. What is the difference between `Statement` and `PreparedStatement`?

Answer: *Statement: SQL string concatenated, parsed each time. PreparedStatement: parameterized, precompiled, prevents SQL injection, better performance for repeated queries.*

Q239. Why is `PreparedStatement` preferred for queries with user input? Explain SQL injection.

Answer: *Parameters bound separately, not concatenated. SQL injection: malicious input like `'; DROP TABLE--` executes as SQL. PreparedStatement treats it as data, not code.*

Q240. What does `ResultSet.next()` return? What happens if you call it when there are no more rows?

Answer: *Returns true if next row exists and cursor moved. Returns false when no more rows. Cursor starts before first row.*

Q241. What is the purpose of `Connection.setAutoCommit(false)`?

Answer: *Disables auto-commit after each statement. Enables manual transaction control. Must explicitly call commit() or rollback().*

Q242. What happens if you don't close a `ResultSet` or `Connection`? What is resource leaking?

Answer: *Resource leak: database connections, cursors remain open. Exhausts connection pool, database resources. Eventually causes failures. Always close in finally or try-with-resources.*

Q243. Can you execute multiple SQL statements with a single `Statement` object?

Answer: *Yes, reuse Statement for multiple executeQuery/executeUpdate calls. But only one active ResultSet per Statement. Close ResultSet before next query.*

Q244. What is the difference between `executeQuery()`, `executeUpdate()`, and `execute()`?

Answer: *executeQuery: SELECT, returns ResultSet. executeUpdate: INSERT/UPDATE/DELETE, returns affected row count. execute: any SQL, returns boolean (true if ResultSet).*

Q245. What does `Class.forName("driver.class")` do? Is it still needed?

Answer: *Loads JDBC driver class, triggers static initializer that registers driver. JDBC 4.0+: automatic via ServiceLoader, not needed.*

Q246. How do you handle transactions in JDBC? What methods are involved?

Answer: *setAutoCommit(false), execute statements, commit() on success, rollback() on error. Wrap in try-catch, rollback in catch block.*

Q247. User enters: `' OR '1'='1`. How does this become SQL injection with Statement? How does PreparedStatement prevent it?

Answer: *Statement: SELECT * FROM users WHERE name=" OR '1'='1' returns all rows. PreparedStatement: treats entire input as literal string value, not SQL code.*

Q248. Write JDBC code to get count of employees with salary > 50000 using PreparedStatement.

Answer: *PreparedStatement ps = conn.prepareStatement("SELECT COUNT(*) FROM employees WHERE salary > ?"); ps.setInt(1, 50000); ResultSet rs = ps.executeQuery(); rs.next(); int count = rs.getInt(1).*

17 Entity-Relationship Diagrams (ERD)

Q249. What is the difference between an entity and an attribute? When should something be an entity vs. an attribute?

Answer: *Entity: independent thing with identity (Student). Attribute: property of entity (name). Make entity if: has own attributes, involved in relationships, multiple values needed.*

Q250. Explain 1:1, 1:N, and M:N cardinality with examples. How is M:N implemented in tables?

Answer: *1:1: Person-Passport. 1:N: Department-Employees. M:N: Students-Courses. M:N needs junction table with FKs to both entities.*

Q251. What is a weak entity? What is a partial key?

Answer: *Weak entity: cannot be uniquely identified by own attributes, depends on owner entity. Partial key: attribute that's unique only within owner (Room number within Building).*

Q252. What is a composite attribute? What is a multivalued attribute? How do you handle each in tables?

Answer: *Composite: attribute with sub-parts (Address: street, city). Flatten to columns. Multivalued: multiple values (phone numbers). Separate table with FK.*

Q253. What is total vs. partial participation in a relationship?

Answer: *Total: every entity instance must participate (every employee MUST belong to dept). Partial: participation optional (employee MAY have projects).*

Q254. What is a derived attribute? Should it be stored in the database?

Answer: *Derived: calculated from other attributes (age from birthdate). Generally don't store; calculate when needed. Store if calculation expensive and data stable.*

Q255. What is the difference between identifying and non-identifying relationships?

Answer: *Identifying: child's PK includes parent's PK, child cannot exist without parent (OrderItem needs Order). Non-identifying: child has own PK, FK is not part of PK.*

Q256. How do you represent recursive relationships in an ERD?

Answer: *Entity related to itself. Line from entity back to itself. Example: Employee manages Employee. Implemented as self-referencing FK (manager_id references employee_id).*

Q257. Convert this to tables: Student(sid, name) and Course(cid, title) with M:N relationship Enrolls(grade).

Answer: *Three tables: Student(sid PK, name), Course(cid PK, title), Enrollment(sid FK, cid FK, grade, PK(sid,cid)). Junction table has composite PK.*

Q258. A Room belongs to exactly one Building. Room 101 exists in multiple buildings. How do you model this?

Answer: *Room is weak entity. PK is (building_id, room_number). room_number is partial key, unique only within a building.*

18 Transactions and ACID

Q259. What do ACID properties stand for? Explain each briefly.

Answer: *Atomicity: all or nothing. Consistency: valid state to valid state. Isolation: concurrent transactions don't interfere. Durability: committed changes survive failures.*

Q260. What is the difference between `COMMIT` and `ROLLBACK`?

Answer: *COMMIT: makes transaction changes permanent. ROLLBACK: undoes all changes since transaction start. Transaction ends with either.*

Q261. A transaction does INSERT, UPDATE, then fails on DELETE. What happens with ROLLBACK?

Answer: *All three operations undone. Database returns to state before transaction started. Atomicity ensures all-or-nothing.*

19 Syntax Error Identification

For each SQL statement below, identify the specific error and explain how to fix it.

Q262. Find the error: This query attempts to count employees per department but fails.

```
SELECT department, COUNT(*)
FROM employees
WHERE salary > 50000;
```

Answer: *Error: Missing GROUP BY clause. When using COUNT(*) with a non-aggregated column (department), must add GROUP BY department.*

Q263. Find the error: This query tries to filter by date range but has invalid syntax.

```
SELECT * FROM orders
WHERE order_date BETWEEN '2024-01-01' AND;
```

Answer: *Error: Incomplete BETWEEN clause. BETWEEN requires two values. Fix: BETWEEN '2024-01-01' AND '2024-12-31'.*

Q264. Find the error: This query tries to find above-average salaries but uses aggregate incorrectly.

```
SELECT name, salary
FROM employees
WHERE salary > AVG(salary);
```

Answer: *Error: Aggregate function AVG() in WHERE clause. WHERE executes before grouping. Fix: Use subquery: WHERE salary > (SELECT AVG(salary) FROM employees).*

Q265. Find the error: This UPDATE uses a subquery that could return multiple values.

```
UPDATE employees
SET department_id = (SELECT id FROM departments);
```

Answer: *Error: Subquery returns multiple rows but scalar expected. Fix: Add WHERE to subquery to return single value, e.g., WHERE name = 'Sales'.*

Q266. Find the error: This query mixes DISTINCT with aggregation incorrectly.

```
SELECT DISTINCT department, COUNT(*)
FROM employees
GROUP BY department;
```

Answer: *Error: DISTINCT is redundant with GROUP BY (groups are already unique). Not a syntax error but indicates misunderstanding. Remove DISTINCT.*

Q267. Find the error: This query uses HAVING without proper grouping.

```
SELECT e.name, d.name
FROM employees e, departments d
WHERE e.dept_id = d.id
HAVING COUNT(*) > 5;
```

Answer: *Error: HAVING without GROUP BY. HAVING filters groups, but no groups defined. Fix: Add GROUP BY e.name, d.name or remove HAVING and use different logic.*

Q268. Find the error: This multi-row INSERT has inconsistent value counts.

```
INSERT INTO employees (id, name, salary)
VALUES (1, 'John', 50000),
       (2, 'Jane');
```

Answer: *Error: Second row has 2 values but 3 columns specified. All rows must have same column count. Fix: (2, 'Jane', 60000).*

Q269. Find the error: This IN subquery returns wrong number of columns.

```
SELECT name FROM employees
WHERE department_id IN
    (SELECT id, name FROM departments);
```

Answer: *Error: IN subquery returns 2 columns but expects 1. Fix: Remove `'name'` from subquery: SELECT id FROM departments.*

Q270. Find the error: This CREATE TABLE references a column that doesn't exist.

```
CREATE TABLE orders (
    id INT PRIMARY KEY,
    amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

Answer: *Error: customer_id column not defined in table but referenced in FK. Fix: Add column definition: customer_id INT before the FK constraint.*

Q271. Find two errors: This query has multiple issues with aggregates and grouping.

```
SELECT name, MAX(salary)
FROM employees
WHERE MAX(salary) > 100000
GROUP BY department;
```

Answer: *Two errors: 1) MAX() in WHERE (use HAVING). 2) `'name'` not in GROUP BY. Fix: Add name to GROUP BY and change WHERE to HAVING MAX(salary) > 100000.*

Q272. Find the error: This DELETE statement is missing a keyword.

```
DELETE employees
WHERE hire_date < '2020-01-01';
```

Answer: *Error: Missing FROM keyword. Correct syntax: DELETE FROM employees WHERE hire_date < `'2020-01-01'`.*

Q273. Find the logic error: This LEFT JOIN may not return expected results.

```
SELECT * FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id
WHERE d.name = 'Sales' OR d.name IS NULL;
```

Answer: *Logic error: WHERE d.name = `'Sales'` filters out unmatched rows first, then OR d.name IS NULL brings back employees with no dept. Ambiguous intent. Move `'Sales'` condition to ON clause if you want all employees plus dept info.*

Q274. Find the error: This ORDER BY references a non-existent column position.

```
SELECT name, salary
FROM employees
ORDER BY 3;
```

Answer: *Error: ORDER BY 3 references column position 3, but only 2 columns in SELECT. Fix: Use ORDER BY 1 or ORDER BY 2, or use column name.*

Q275. Find the error: This HAVING clause references a non-aggregated column.

```
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id
HAVING salary > 50000;
```

Answer: *Error: HAVING references `'salary'` which is neither in GROUP BY nor aggregated. Fix: HAVING AVG(salary) > 50000.*

Q276. Find the error: This CREATE TABLE has a duplicate constraint.

```
CREATE TABLE test (
    id INT PRIMARY KEY PRIMARY KEY
);
```

Answer: *Error: PRIMARY KEY specified twice. A table can have only one primary key. Fix: Remove one PRIMARY KEY keyword.*

Q277. Find the error: This query compares with NULL incorrectly.

```
SELECT * FROM employees
WHERE hire_date = NULL;
```

Answer: *Error: Cannot use = with NULL (result is always UNKNOWN). Fix: Use IS NULL: WHERE hire_date IS NULL.*

Q278. Find the potential runtime error: This UPDATE's subquery could fail.

```
UPDATE employees
SET salary = salary * 1.1
WHERE department_id =
    (SELECT department_id FROM employees WHERE name = 'John');
```

Answer: *Potential error: If multiple employees named `'John'` exist, subquery returns multiple rows causing error. Fix: Use IN instead of =, or add LIMIT 1, or use more specific WHERE.*

Q279. Find the error: This UNION has mismatched column counts.

```
SELECT name, department
FROM employees
UNION
SELECT name FROM contractors;
```

Answer: *Error: First SELECT has 2 columns, second has 1. UNION requires same column count. Fix: Add second column to second query or remove from first.*

Q280. Find the potential issue: This INSERT doesn't specify columns.

```
INSERT INTO employees
VALUES (1, 'John', 50000, 'Sales');
```

Answer: *Potential issue: No column list means values must match table column order exactly. If table structure changes, INSERT breaks. Also may have wrong number of values. Fix: Explicitly list columns.*

Q281. Find the error: This scalar subquery could return multiple rows.

```
SELECT e.name,
       (SELECT d.name FROM departments d)
FROM employees e;
```

Answer: *Error: Subquery in SELECT must return single value, but this returns all department names. Fix: Add WHERE d.id = e.dept_id to correlate with outer query.*

Q282. Find the incomplete syntax: This FK reference is missing information.

```
ALTER TABLE employees
ADD COLUMN manager_id INT REFERENCES employees;
```

Answer: *Error: Missing column in REFERENCES. Should specify which column: REFERENCES employees(id). Without it, may fail or reference wrong column.*

Q283. Find the error: This BETWEEN clause is incomplete.

```
SELECT name
FROM employees
WHERE salary BETWEEN 50000;
```

Answer: *Error: BETWEEN requires two values with AND. Fix: WHERE salary BETWEEN 50000 AND 100000.*

Q284. Find the error: This ORDER BY tries to aggregate an alias.

```
SELECT department, SUM(salary) as total
FROM employees
GROUP BY department
ORDER BY SUM(total);
```

Answer: *Error: SUM(total) tries to aggregate the alias `'total'` which is already a SUM. Fix: ORDER BY total or ORDER BY SUM(salary).*

Q285. Find the potential issue: This view definition may not work in all databases.

```
CREATE VIEW emp_view AS
SELECT * FROM employees
ORDER BY salary DESC;
```

Answer: *Potential issue: Standard SQL doesn't allow ORDER BY in views without LIMIT. Views represent unordered sets. ORDER BY may be ignored. Fix: Remove ORDER BY from view, apply in query.*

Q286. Debug this query: Task: Find employees hired after 2020. Query returns 0 rows but table has data.

```
SELECT * FROM employees
WHERE hire_date > 2020;
```

Answer: *Error: Comparing date to integer 2020. Should compare to date string. Fix: WHERE hire_date > '2020-12-31' or WHERE EXTRACT(YEAR FROM hire_date) > 2020.*

Q287. Debug this query: Task: Find names containing `'son'`. Query returns no results but `'Johnson'` exists.

```
SELECT * FROM employees
WHERE name LIKE 'son';
```

Answer: *Error: Missing wildcards. LIKE 'son' matches exactly 'son' only. Fix: WHERE name LIKE '%son%' to find 'son' anywhere in name.*

Q288. Debug this query: Task: Find employees with NULL department. Query returns empty.

```
SELECT * FROM employees
WHERE department_id = NULL;
```

Answer: *Error: Using = with NULL. NULL = NULL is UNKNOWN, not TRUE. Fix: WHERE department_id IS NULL.*

Q289. Debug this query: Task: Find employees NOT in department 5 or 10. Employees with NULL dept_id are missing.

```
SELECT * FROM employees
WHERE department_id NOT IN (5, 10);
```

Answer: *Issue: NOT IN excludes NULLs. NULL NOT IN (...) is UNKNOWN. Fix: WHERE (department_id NOT IN (5,10) OR department_id IS NULL).*

Q290. Debug this query: Task: Count employees per department. Shows wrong count for some departments.

```
SELECT d.name, COUNT(*)
FROM departments d
LEFT JOIN employees e ON d.id = e.dept_id
GROUP BY d.name;
```

Answer: *Issue: COUNT(*) counts 1 even for departments with no employees (the NULL row). Fix: COUNT(e.id) returns 0 for empty departments.*

Q291. Debug this query: Task: Get second highest salary. Query sometimes returns wrong value.

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Answer: *Issue: If multiple employees have max salary, this still works. But if only 1 employee exists, returns NULL. Also fails with ties at second place. Consider DISTINCT or OFFSET/LIMIT.*

Q292. Debug this query: Task: Find employees earning above average. Returns different results each run.

```
SELECT name FROM employees e1
WHERE salary > (SELECT AVG(salary) FROM employees e2
                WHERE e2.dept_id = e1.dept_id);
```

Answer: *Not an error - this compares to department average (correlated). If task wants overall average, remove WHERE clause from subquery.*

Q293. Debug this query: Task: List all departments with their employee count, sorted by count descending.

```
SELECT department_id, COUNT(*) as cnt
FROM employees
GROUP BY department_id
ORDER BY COUNT(*) DESC;
```

Answer: *Missing departments with 0 employees. Fix: Start from departments table with LEFT JOIN: SELECT d.id, COUNT(e.id) FROM departments d LEFT JOIN employees e ON ...*

Q294. Debug this query: Task: Find products priced between \$10 and \$50. Some \$10 and \$50 items missing.

```
SELECT * FROM products
WHERE price > 10 AND price < 50;
```

Answer: *Error: Using > and < excludes boundary values. Fix: WHERE price >= 10 AND price <= 50, or use BETWEEN 10 AND 50 (inclusive).*

Q295. Debug this query: Task: Calculate average salary per department. Wrong average for some.

```
SELECT department_id, SUM(salary)/COUNT(*)
FROM employees
GROUP BY department_id;
```

Answer: *Issue: If salary has NULLs, COUNT(*) counts all rows but SUM ignores NULLs, giving wrong average. Fix: Use AVG(salary) which handles NULLs correctly.*

Q296. Debug this query: Task: Find duplicate emails. Query returns all rows, not just duplicates.

```
SELECT email, COUNT(*)
FROM users
GROUP BY email;
```

Answer: *Missing HAVING clause. This returns all emails with their counts. Fix: Add HAVING COUNT(*) > 1 to filter only duplicates.*

Q297. Debug this query: Task: Show employee and manager names. Employees without managers don't appear.

```
SELECT e.name, m.name as manager
FROM employees e
JOIN employees m ON e.manager_id = m.id;
```

Answer: *Using INNER JOIN excludes employees with NULL manager_id. Fix: Use LEFT JOIN to include employees without managers (manager shows as NULL).*

Q298. Debug this query: Task: Find orders from last 7 days. Returns no rows but recent orders exist.

```
SELECT * FROM orders
WHERE order_date > CURRENT_DATE - 7;
```

Answer: *Issue: CURRENT_DATE - 7 subtracts integer (days) which works in PostgreSQL. But if order_date is TIMESTAMP, may miss same-day orders. Use INTERVAL '7 days' for clarity.*

Q299. Debug this query: Task: Concatenate first and last name. Some results show NULL.

```
SELECT first_name || ' ' || last_name as full_name
FROM employees;
```

Answer: *Issue: || operator propagates NULL. If either name is NULL, result is NULL. Fix: COALESCE(first_name,") || ' ' || COALESCE(last_name,"), or use CONCAT().*

Q300. Debug this query: Task: Find the 3rd highest salary. Returns wrong value.

```
SELECT salary FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

Answer: *Issue: If multiple employees have same salary, this may skip ranks. For true 3rd highest distinct salary: SELECT DISTINCT salary ... or use DENSE_RANK().*

Q301. Debug this query: Task: Update all prices by 10%. Some prices unchanged.

```
UPDATE products
SET price = price * 1.10
WHERE category = 'Electronics';
```

Answer: *Not a bug if intentional. But if task is ALL prices, the WHERE clause limits update. Remove WHERE to update all rows. Also NULL prices stay NULL.*

Q302. Debug this query: Task: Find names starting with vowels (A,E,I,O,U). Missing lowercase matches.

```
SELECT name FROM employees
WHERE name LIKE 'A%' OR name LIKE 'E%'
   OR name LIKE 'I%' OR name LIKE 'O%'
   OR name LIKE 'U%';
```

Answer: *Issue: LIKE is case-sensitive.* `'alice'` *won't match* `'A%'`*. Fix: Use ILIKE (PostgreSQL) or add lowercase patterns, or WHERE UPPER(name) LIKE* `'A%'` *OR ...*

20 Misleading Queries (Looks Correct But Wrong)

Each query below appears correct but produces wrong results. Identify the logical flaw.

Q303. Task: Find employees who earn more than everyone in the Sales department.

```
SELECT * FROM employees
WHERE salary > (SELECT salary FROM employees
                WHERE department = 'Sales');
```

Answer: *Flaw: Subquery returns multiple rows, causing error. Even if it worked, comparing to one salary isn't "more than everyone". Fix: WHERE salary > (SELECT MAX(salary) FROM employees WHERE department =* `'Sales'`*).*

Q304. Task: Find departments where ALL employees earn more than 50000.

```
SELECT department_id FROM employees
WHERE salary > 50000
GROUP BY department_id;
```

Answer: *Flaw: Returns departments that have AT LEAST ONE employee earning > 50000, not ALL. Fix: Use NOT EXISTS to check no employee earns <= 50000, or compare COUNT(*) with COUNT(CASE WHEN salary > 50000...).*

Q305. Task: Find customers who have placed orders for ALL products.

```
SELECT DISTINCT customer_id FROM orders
WHERE product_id IN (SELECT id FROM products);
```

Answer: *Flaw: Returns customers who ordered ANY product, not ALL products. Fix: GROUP BY customer_id HAVING COUNT(DISTINCT product_id) = (SELECT COUNT(*) FROM products).*

Q306. Task: Find employees whose salary is above the average of their department.

```
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Answer: *Flaw: Compares to company-wide average, not department average. Fix: Add correlation: WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id = employees.dept_id).*

Q307. Task: Count how many employees have no orders assigned.

```
SELECT COUNT(*) FROM employees e
LEFT JOIN orders o ON e.id = o.employee_id
WHERE o.id = NULL;
```

Answer: *Flaw: o.id = NULL is always UNKNOWN, returns 0. Fix: WHERE o.id IS NULL. Common mistake mixing = with NULL.*

Q308. Task: Find products that have never been ordered.

```
SELECT * FROM products
WHERE id NOT IN (SELECT product_id FROM orders);
```

Answer: *Flaw: If orders.product_id contains any NULL, NOT IN returns empty set. Fix: Add WHERE product_id IS NOT NULL in subquery, or use NOT EXISTS.*

Q309. Task: Find the department with the highest total salary.

```
SELECT department_id, SUM(salary) as total
FROM employees
GROUP BY department_id
ORDER BY total DESC;
```

Answer: *Flaw: Returns ALL departments sorted, not just the highest. Missing LIMIT 1. Also if two depts tie, only shows one.*

Q310. Task: Delete duplicate rows, keeping one copy of each.

```
DELETE FROM employees
WHERE id NOT IN (SELECT MIN(id) FROM employees
                 GROUP BY name, email);
```

Answer: *Flaw: Looks correct but fails in MySQL (can't select from table being deleted). PostgreSQL works. Also assumes id is unique identifier for "which to keep".*

Q311. Task: Find employees who joined in the same month as their manager.

```
SELECT e.name FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE EXTRACT(MONTH FROM e.hire_date) =
      EXTRACT(MONTH FROM m.hire_date);
```

Answer: *Flaw: Compares month only, ignoring year. Jan 2020 matches Jan 2015. Fix: Also compare YEAR, or compare DATE_TRUNC('month', date).*

Q312. Task: Find the employee with the longest name.

```
SELECT * FROM employees
WHERE LENGTH(name) = (SELECT MAX(LENGTH(name))
                      FROM employees);
```

Answer: *Flaw: Returns multiple rows if tie exists. May or may not be intended. If only one needed: add LIMIT 1. Query is correct if ties should all appear.*

Q313. **Task:** Increase salary by 10% for employees who earn below average.

```
UPDATE employees
SET salary = salary * 1.1
WHERE salary < (SELECT AVG(salary) FROM employees);
```

Answer: *Flaw: Query is actually correct in PostgreSQL (subquery evaluated once before update). But in some DBs, average recalculates during update causing inconsistent results.*

Q314. **Task:** Find pairs of employees in the same department.

```
SELECT e1.name, e2.name
FROM employees e1, employees e2
WHERE e1.department_id = e2.department_id;
```

Answer: *Flaw: Includes self-pairs (John, John) and duplicate pairs (John, Jane) and (Jane, John). Fix: Add e1.id < e2.id to get unique pairs without self-matching.*

Q315. **Task:** Find orders placed on weekends.

```
SELECT * FROM orders
WHERE EXTRACT(DOW FROM order_date) = 6
   OR EXTRACT(DOW FROM order_date) = 7;
```

Answer: *Flaw: PostgreSQL DOW is 0-6 (Sunday=0, Saturday=6). There is no 7. Fix: DOW = 0 OR DOW = 6 for Sunday and Saturday.*

Q316. **Task:** Find customers who spent more than $1000 total.

```
SELECT customer_id FROM orders
WHERE SUM(amount) > 1000
GROUP BY customer_id;
```

Answer: *Flaw: SUM() in WHERE clause is invalid. Aggregates can't be in WHERE. Fix: Move condition to HAVING: GROUP BY customer_id HAVING SUM(amount) > 1000.*

Q317. **Task:** Find employees not assigned to any project.

```
SELECT * FROM employees e
WHERE e.id != ALL (SELECT employee_id
                   FROM project_assignments);
```

Answer: *Flaw: If project_assignments has NULL employee_id, != ALL returns empty (NULL comparison). Fix: Add WHERE employee_id IS NOT NULL, or use NOT EXISTS.*

Q318. **Task:** Get each employee's rank by salary within their department.

```
SELECT name, department_id,
       (SELECT COUNT(*) FROM employees e2
        WHERE e2.department_id = e1.department_id
        AND e2.salary > e1.salary) + 1 as rank
FROM employees e1;
```

Answer: *Flaw: Ties get same rank but next rank skips (1,1,3 not 1,1,2). This is RANK() behavior. If DENSE_RANK() wanted, need different logic. Also counts those with higher salary, not >= .*

Q319. **Task:** Find the most recent order for each customer.

```
SELECT customer_id, MAX(order_date), order_id
FROM orders
GROUP BY customer_id;
```

Answer: *Flaw: order_id not in GROUP BY and not aggregated. PostgreSQL error. MySQL returns arbitrary order_id, not the one with MAX date. Fix: Use subquery or window function.*

Q320. **Task:** Calculate percentage of employees in each department.

```
SELECT department_id,
       COUNT(*) / (SELECT COUNT(*) FROM employees) * 100
FROM employees
GROUP BY department_id;
```

Answer: *Flaw: Integer division truncates. 3/10 = 0, not 0.3. Fix: Cast to decimal: COUNT(*)::DECIMAL / ... or multiply by 100.0 first.*

Q321. **Task:** Find employees who have worked on projects with employees from other departments.

```
SELECT DISTINCT e1.name
FROM employees e1
JOIN project_assignments p1 ON e1.id = p1.employee_id
JOIN project_assignments p2 ON p1.project_id = p2.project_id
JOIN employees e2 ON p2.employee_id = e2.id
WHERE e1.department_id != e2.department_id;
```

Answer: *Flaw: Looks correct but fails if department_id is NULL for any employee - != with NULL is UNKNOWN. Fix: Add AND e1.department_id IS NOT NULL AND e2.department_id IS NOT NULL.*

Q322. **Task:** Find average order value, excluding orders over $1000.

```
SELECT AVG(amount) FROM orders
HAVING amount <= 1000;
```

Answer: *Flaw: HAVING without GROUP BY, and HAVING can't reference non-aggregated column like this. Fix: Use WHERE amount <= 1000 (before aggregation).*

Q323. **Task:** Find customers whose total orders exceed the average customer's total.

```
SELECT customer_id, SUM(amount)
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > AVG(SUM(amount));
```

Answer: *Flaw: Can't nest aggregates like AVG(SUM()). Fix: Use subquery: HAVING SUM(amount) > (SELECT AVG(total) FROM (SELECT SUM(amount) as total FROM orders GROUP BY customer_id) t).*

Q324. **Task:** Update employee's department to match their manager's department.

```
UPDATE employees e
SET department_id = (SELECT department_id
                     FROM employees
                     WHERE id = e.manager_id);
```

Answer: *Flaw: Employees with NULL manager_id get department set to NULL (subquery returns NULL). May not be intended. Fix: Add WHERE e.manager_id IS NOT NULL to UPDATE.*

Q325. **Task:** Find products ordered by every customer.

```
SELECT product_id FROM orders
GROUP BY product_id
HAVING COUNT(DISTINCT customer_id) =
       (SELECT COUNT(DISTINCT customer_id) FROM orders);
```

Answer: *Flaw: Compares to customers who ordered SOMETHING, not ALL customers. Fix: Compare to (SELECT COUNT(*) FROM customers) for total customer count.*

Q326. **Task:** Find the second most expensive product in each category.

```
SELECT category, name, price
FROM products p1
WHERE price = (SELECT MAX(price) FROM products p2
               WHERE p2.category = p1.category
               AND p2.price < (SELECT MAX(price) FROM products p3
                               WHERE p3.category = p1.category));
```

Answer: *Flaw: If category has only 1 product, inner subquery returns NULL, no match. Also returns multiple if ties exist at second place. May need LIMIT or DENSE_RANK approach.*

Q327. **Task:** Find employees hired in the last 30 days of their hire year.

```
SELECT * FROM employees
WHERE hire_date >= CURRENT_DATE - INTERVAL '30 days';
```

Answer: *Flaw: Finds employees hired in last 30 days from TODAY, not last 30 days of THEIR hire year. Fix: WHERE EXTRACT(DOY FROM hire_date) >= 336 (roughly Dec 1+).*

Q328. **Task:** Count orders per month for 2024.

```
SELECT EXTRACT(MONTH FROM order_date) as month, COUNT(*)
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2024
GROUP BY EXTRACT(MONTH FROM order_date);
```

Answer: *Flaw: Months with zero orders won't appear. Need to generate all 12 months and LEFT JOIN. Query only shows months that have data.*

Q329. **Task:** Find employees whose name appears more than once (different people, same name).

```
SELECT name FROM employees
GROUP BY name
HAVING COUNT(*) > 1;
```

Answer: *Flaw: Query is correct! But only returns names, not the actual duplicate rows. To get full rows: use this as subquery in WHERE name IN (...).*

Q330. **Task:** Give a 5% raise to employees who earn less than their department average.

```
UPDATE employees e1
SET salary = salary * 1.05
WHERE salary < (SELECT AVG(salary) FROM employees e2
                WHERE e2.department_id = e1.department_id);
```

Answer: *Flaw: Employees in single-person departments compare to themselves. If alone, salary = avg, so no raise. May not be intended behavior.*

Q331. **Task:** Find the total revenue per product category.

```
SELECT p.category, SUM(o.quantity * p.price)
FROM products p
JOIN orders o ON p.id = o.product_id
GROUP BY p.category;
```

Answer: *Flaw: If order has its own price (price at time of order), using p.price uses CURRENT price, not historical. Revenue calculation may be wrong if prices changed.*

Q332. **Task:** Find employees who report to someone in a different city.

```
SELECT e.name FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.city <> m.city;
```

Answer: *Flaw: If either city is NULL, <> returns UNKNOWN, row excluded. Employees with NULL city won't appear even if manager has different city. Fix: Handle NULLs with COALESCE or IS DISTINCT FROM.*